

Spring Framework

Introduction of Spring :-

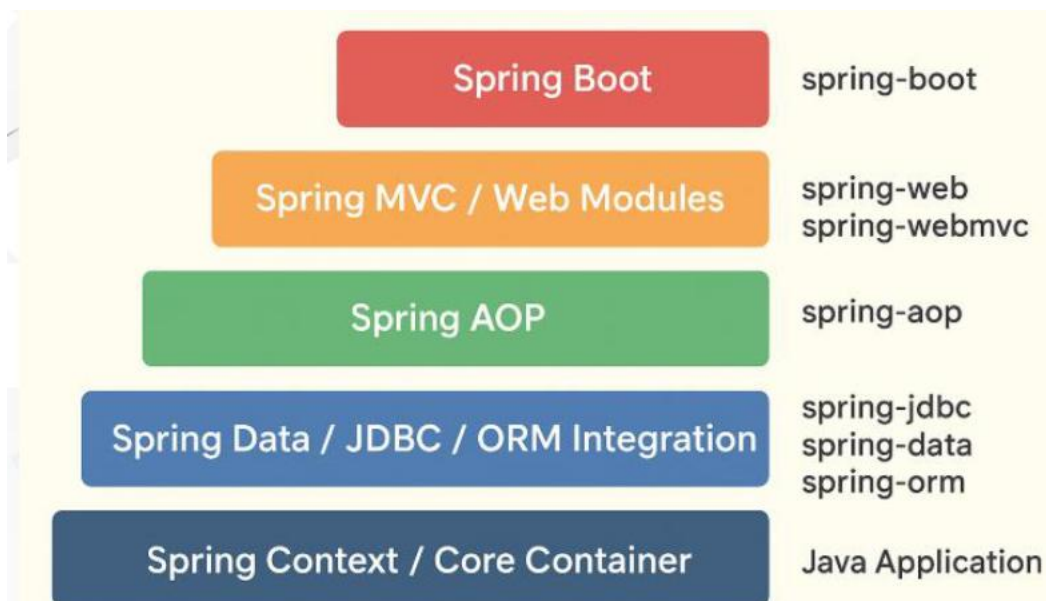
- Spring Framework is a lightweight and open-source framework used to develop enterprise-level Java applications.
- It was developed by Rod Johnson in 2003 to simplify the complexity of Java EE development.
- It is based on the concepts of Inversion of Control (IoC) and Dependency Injection (DI).
- Spring helps to create flexible, modular, and easily maintainable applications.

Features of Spring Framework :-



Architecture of Spring Framework :-

- The Spring Framework architecture is layered and modular.
- It allows developers to use only the required parts of the framework.
- The architecture is made up of several core modules.



There are five layers of spring framework are as follows:

1. Spring Context / Core Container (Base Layer)

- It is the foundation layer of the Spring Framework.
- It provides IoC and DI features and manages the lifecycle of beans.
- Main packages: spring-core, spring-beans, spring-context.

2. Spring Data / JDBC / ORM Integration Layer

- This layer handles data access and database integration.
- It works with JDBC, Hibernate, and JPA for database operations.
- Modules: spring-jdbc, spring-data, spring-orm.

3. Spring AOP (Aspect-Oriented Programming) Layer

- AOP enables handling cross-cutting concerns like logging and security.
- It allows adding such features without changing the main code.
- Module: spring-aop.

4. Spring MVC / Web Modules Layer

- This layer supports web and REST application development.
- It follows the MVC pattern for clean separation of logic and view.
- Modules: spring-web, spring-webmvc.

5. Spring Boot (Top Layer)

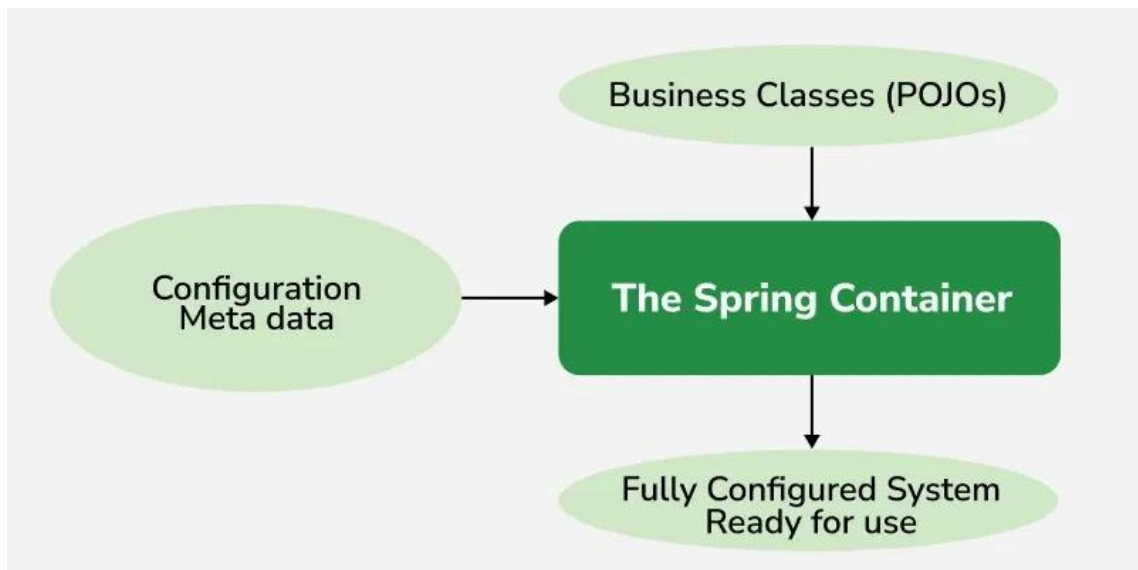
- Spring Boot simplifies configuration and setup of Spring applications.
- It provides auto-configuration, embedded servers, and ready templates.
- Module: spring-boot.

Spring Framework Module :-

Layer	Function	Modules
Core Container	Handles IoC and bean management.	spring-core, spring-beans, spring-context, spring-expression
Data Access	Manages database and transaction support.	spring-jdbc, spring-orm, spring-tx, spring-jms
AOP	Manages cross-cutting concerns.	spring-aop
Web	Supports web and MVC applications.	spring-web, spring-webmvc
Test	Provides testing support.	spring-test

Spring IoC Container (Inversion of Control) :-

- Inversion of Control (IoC) is the main principle of the Spring Framework.
- It means the control of creating and managing objects is given to the Spring container, not the developer.
- The container creates, configures, and manages the lifecycle of objects called beans.
- IoC is mainly implemented using Dependency Injection (DI) in Spring.
- It helps in achieving loose coupling, easy testing, and better maintainability of code.



Why IoC ?

- Loose Coupling — classes don't depend directly on concrete implementations
- Easier Testing — objects can be replaced with mock dependencies
- Better Code Management — Spring handles the lifecycle of beans automatically.

Types of Spring Container :-

There are two types of spring container are as follows:



1. BeanFactory :-

- BeanFactory is the simplest container in Spring used to manage beans.
- It provides the basic support for Dependency Injection (DI).
- Beans are created lazily, i.e., only when they are requested.
- It is suitable for lightweight applications or memory-constrained devices.

Declaration:

```
BeanFactory factory = new XmlBeanFactory(new  
FileSystemResource("applicationContext.xml"));
```

2. ApplicationContext :-

- ApplicationContext is the advanced container in Spring.
- It provides all BeanFactory features plus AOP, events, and message resource handling.
- Beans are created eagerly at the time of container startup.
- It is commonly used in enterprise-level Spring applications.

Declaration:

```
ApplicationContext context = new  
ClassPathXmlApplicationContext("applicationContext.xml");
```

Difference Between BeanFactory & ApplicationContent :-

Feature	BeanFactory	ApplicationContext
Initialization	Lazy (beans created on demand).	Eager (beans created at startup).
Memory Usage	Lightweight.	Heavier due to additional features.
Enterprise Features	No support for AOP, events, i18n, etc.	Full support for enterprise features.
Annotation Support	Limited (requires manual configuration).	Built-in support for annotations.
Use Case	Resource-constrained environments.	Modern applications, especially Spring Boot.

Steps To Create Spring Application :-

1. Create the Java Project.
2. Add spring jar files
3. Create Java class
4. Create the xml file
5. Create the Main method class

Steps :

1. Create a new **Java Project**
2. Create bean/model file in your package like **demo/Student.java**

```
package demo;

public class Student {
    private String name;
    private String address;

    //generate getter & setters

    public void displayinfo() {
        System.out.println(name+" "+address);
    }
}
```

3. Create **applicationContext.xml** file in **src** package

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">

    <bean id="s1" class="demo.Student">
        <property name="name" value="Abcd "></property>
        <property name="address" value="Pune "></property>
    </bean>

</beans>
```

4. Create **Test.java** with **main method**

```
ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");  
  
Student s = (Student) context.getBean("s1");  
s.displayinfo();
```

OR

```
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("applicationContext.xml"));  
Student s = (Student) factory.getBean("s1");  
s.display();
```

Implementation of Spring Application :-

Student.java –

```
package Demo;

public class Student {
    private String name;
    private String address;

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }

    public void showdetails()
    {
        System.out.println("Name= "+name);
        System.out.println("Address= "+address);
    }
}
```

applicationContext.xml –

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="s1" class="Demo.Student">
        <property name="name" value="abcd" />
        <property name="address" value="pune" />
    </bean>

    <bean id="s2" class="Demo.Student">
        <property name="name" value="xyz" />
        <property name="address" value="pune" />
    </bean>

</beans>
```

Test.java –

```
package Demo;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {
        ApplicationContext con = new ClassPathXmlApplicationContext("applicationContext.xml");
        Student s = (Student) con.getBean("s1");
        s.showdetails();
    }
}
```

Output :-

```
log4j:WARN No appenders could be found for logger (org.springframework.context.support.ClassPathXmlApplicationContext).
log4j:WARN Please initialize the log4j system properly.
Name= abcd
Address= pune
```